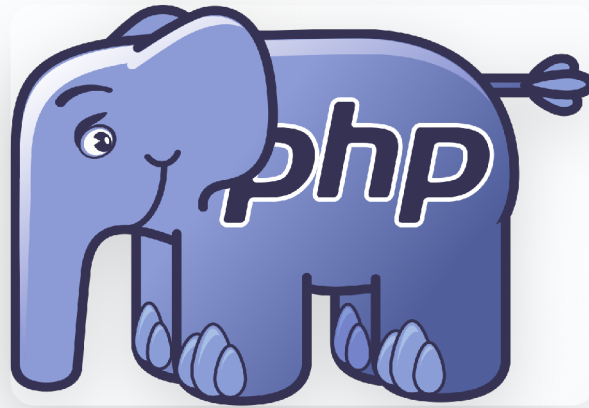


Développement d'un site web dynamique

bases



On parle de quoi ?

- 1 Syntaxe
- 2 Sorties
- 3 Erreurs
- 4 Programmation modulaire
- 5 Déclaration d'une variable
- 6 Affectation d'une variable
- 7 Type d'une variable
- 8 Constantes
- 9 Les instructions

Syntaxe

- **[idem Javascript]** Tous les blancs (espaces, tabulations, retours à la ligne) dans le code sont ignorés (pas d'excuse pour ne pas écrire un beau code bien indenté !)
- **[idem Javascript]** Deux formats pour les commentaires :
 - `//` pour tout le reste de la ligne
 - `/* ... /*` pour plusieurs lignes ou une partie de ligne
- Chaque instruction PHP doit se terminer par un `;`. Le dernier avant la balise fermante peut être omis.

Syntaxe

- Le nom d'une variable commence par **\$**

```
$maVariable = 1;
```

- Il doit commencer par une lettre ou **_** et être composé de lettres, de chiffres et/ou de **_**.

```
// correct  
$_maVar = 1;  
// correct  
$maVar_ = 2;  
// pas correct  
$1var = "Hello !";
```


Syntaxe

- **[idem Javascript]** Les noms de variables PHP sont sensibles à la casse
- Les noms des fonctions, classes et les mots-clés ne sont **pas** sensibles à la casse

Sorties

Sortie : contenu imprimé (intégrée) directement dans le document HTML.

→ Fonction `echo` qui imprime une chaîne de caractère

```
echo "Bonjour ";  
$prenom = "lucas";  
echo $prenom;  
echo " !";
```

```
Bonjour lucas !
```

Sorties

→ Fonction `print_r` qui affiche le contenu d'une variable

```
$tab = array("red", "green", "blue");  
echo $tab;
```

```
Array ( [0] => red [1] => green [2] => blue )
```

Sorties

→ Fonction `var_dump` qui affiche le type et la valeur d'une variable

```
$a = 32;  
var_dump($a);  
$b = "Salut !";  
var_dump($b);  
$c = 32.5;  
var_dump($c);  
$d = array("red", "green", "blue");  
var_dump($d);
```

Sorties

→ Fonction `var_dump` qui affiche le type et la valeur d'une variable

```
int(32)
string(7) "Salut !"
float(32.5)
array(3) {
    [0]=> string(3) "red"
    [1]=> string(5) "green"
    [2]=> string(4) "blue" }
```

Erreurs

En PHP, il existe trois niveaux d'erreur :



ERROR

L'exécution s'interrompt

Exemple :

oubli d'un point-virgule



WARNING

L'exécution se poursuit

Exemple :

modifier une constante

Programmation modulaire

En JavaScript

Option 1 : directement dans le document HTML.

```
<body>
  <h1>Exemple</h1>
  <p id="para"></p>
  <script>
    document.getElementById("para").innerHTML = "Vous savez, moi..."
  </script>
</body>
```

Programmation modulaire

En JavaScript

Option 2 : dans un fichier qui ne contient que du JavaScript.

```
<body>  
  <h1>Exemple</h1>  
  <p id="para"></p>  
  <script type="text/javascript" src="script.js"></script>  
</body>
```


Programmation modulaire

En JavaScript

Le fichier `script.js` contient :

```
document.getElementById("para").innerHTML = "Vous savez, moi..."
```

Programmation modulaire

En PHP

Option 1 : directement dans le document HTML (à l'endroit où le résultat doit être placé)

```
<body>
  <h1>Exemple</h1>
  <p>
    <?php
      echo "Vous savez, moi..."
    ?>
  </p>
</body>
```

Programmation modulaire

En PHP

Option 2 : dans un fichier qui ne contient que du PHP.

```
<body>
  <h1>Exemple</h1>
  <p>
    <?php
      include("situation.php");
    ?>
  </p>
</body>
```

Programmation modulaire

En PHP

Le fichier `situation.php` contient :

```
<?php  
echo "Vous savez, moi..."  
?>
```

Déclaration d'une variable

En PHP, pas besoin de déclarer ! Une variable est déclarée quand on lui donne une valeur pour la première fois.

```
// déclaration de la variable "compteur" et initialisation  
$compteur = 1;
```

On peut utiliser la fonction `isset` pour tester si une variable existe et ne vaut pas `NULL`

```
isset($var); //false  
$var = 1;  
isset($var); //true  
$autreVar = null;  
isset($autreVar); // false
```

Affectation d'une variable

Affectation par valeur

```
$ville = "Beauraing";  
$ville2 = $ville; //ville2 contient "Beauraing"
```

Affectation d'une variable

Affectation avec opération

```
$increment = 2;  
$compteur = 0;  
  
$compteur += $increment;  
$compteur -= $increment;  
$compteur *= $increment;  
$compteur /= $increment;  
$compteur .= $increment;
```

Type d'une variable

PHP est un langage non typé

[idem Javascript] les variables ne sont pas typées :

- pas besoin de déclarer le type des variables / arguments des fonctions
- le type d'une variable peut changer au cours du programme
- **conversion implicite**

```
$var1 = "12";  
$var2 = 2;  
$res1 = $var1 * $var2;  
$res2 = $var1 . $var2;
```


Type d'une variable

La fonction gettype

```
gettype($var); // indique le type actuel de $var
```

Elle peut indiquer (notamment) :

- **boolean** : valeurs booléennes
- **integer** : entiers
- **double** : réels
- **string** : chaîne de caractères
- **boolean** : tableaux
- **NULL** : valeur NULL

Type d'une variable

Nombres

Valeurs

→ entiers (42, -6, 0, ...)

→ réels (1.5, -2.02, ...)

Opérateurs

+ - * / % ++ --

Type d'une variable

Booléens

Valeurs

- `true` ou `TRUE`
- `false` ou `FALSE`
- `0`
- `"`
- `"0"`
- `[]` (tableau vide)
- `{}` (objet vide)
- `NULL`

Type d'une variable

Booléens

Opérateurs

- `!`
- `&&` ou `and`
- `||` ou `or`

Type d'une variable

Absence de valeur

La valeur `NULL` matérialise l'absence de valeur.

Rappel : `isset` permet de tester si une variable possède une valeur.

Aussi : `unset` permet de supprimer une variable.

```
$var = "Hello";  
isset($var) //true  
unset($var)  
isset($var) //false
```

Type d'une variable

Chaines de caractères

Chaines littérales (avec des apostrophes)

→ le contenu est écrit tel quel

```
$variable = "avec un dollar devant";  
echo 'En PHP, les variable s\'écrivent $variable.';
```

```
En PHP, les variable s'écritent $variable.
```

Type d'une variable

Chaines de caractères

Chaines non littérales (avec des guillemets)

→ les variables sont remplacées par leur valeur

```
$variable = "avec un dollar devant";  
echo "En PHP, les variable s'écrivent $variable.";
```

En PHP, les variable s'écrivent avec un dollar devant.

Constantes

Une constante est une variable qui ne peut pas changer de valeur au cours de l'exécution du programme.

On définit une constante à l'aide de `define` :

```
define('MACONSTANTE', 'le contenu de ma constante');
```

Attention : les constantes s'utilisent sans `$` !

```
define('MESSAGE', 'Bienvenue sur mon site internet !');  
echo MESSAGE;
```


Les instructions

Structure if-else

```
if(condition){  
    // bloc  
} else{  
    //bloc  
}
```

Les instructions

Structure switch

```
switch($var){  
    case valeur:  
        //instruction  
        break;  
    case valeur:  
        //instruction  
        break;  
    default:  
        //instruction  
}
```

Les instructions

Boucle while

```
while(condition){  
    //bloc  
}
```

Boucle for

```
for(initialisation, condition, incrémentation){  
    //bloc  
}
```